

DOCUMENTO DE REQUISITOS DE PRODUTO

Detective's Manual: Caso Blackwood

Jogo de Detetive por Turnos — Aplicação Web

Equipe de Desenvolvimento

Dev 1 — Lucas da Silva Santos (Backend: Algoritmos, Controllers, Rotas, Server; Frontend: Issue #7)

Dev 2 — João David Salomão (Modelagem de Dados, Schemas Mongoose, Seed MongoDB)

Dev 3 — Pollyana de Castro Rodrigues (Frontend: Mapa, Coleta de Pistas, Diálogos, Issue #8)

Disciplinas: Modelagem de Dados / Laboratório de Análise de Algoritmos

Versão: 1.0

1. Visão Executiva

Detective's Manual: Caso Blackwood é uma aplicação web de jogo de detetive por turnos, desenvolvida como trabalho prático integrado às disciplinas de Modelagem de Dados e Laboratório de Análise de Algoritmos. O projeto tem como objetivo central demonstrar, em um contexto aplicado, a implementação e o desempenho de estruturas de dados e algoritmos clássicos — MaxHeap, TSP heurístico, Árvore de Decisão e BFS — em um sistema funcional e interativo.

O jogador assume o papel de um detetive convocado à Blackwood Manor após a morte de Lorde Edmund Blackwood. Ao longo da investigação, coleta pistas distribuídas em um grid 10×10, interroga suspeitos por meio de árvores de diálogo e, ao final, acusa o culpado com base nas evidências reunidas. Cada mecânica do jogo está diretamente vinculada a um algoritmo implementado e testado pelo time.

2. Objetivos do Projeto

2.1 Objetivos Acadêmicos

- Implementar estruturas de dados em contexto real: MaxHeap, Árvore de Decisão, BFS e TSP heurístico.
- Demonstrar análise de complexidade aplicada: $O(\log n)$, $O(V+E)$, $O(n^2)$ e $O(n^2 \cdot 2^n)$.
- Projetar e implementar um banco de dados relacional com schemas Mongoose e seed reproduzível.
- Aplicar práticas de engenharia de software: Git Flow, testes unitários com cobertura 100%, documentação técnica.

2.2 Objetivos de Produto

- Entregar uma aplicação web jogável, com frontend React/TypeScript e backend Node.js/Express/MongoDB.
- Garantir que todas as mecânicas do jogo sejam suportadas por algoritmos testados e documentados.
- Manter separação clara de responsabilidades entre camadas (estruturas, controllers, rotas, frontend).

3. Escopo

3.1 Dentro do Escopo

- Grid 10×10 com névoa de guerra baseada em BFS (raio de visão de 3 células).
- Inventário de pistas gerenciado por MaxHeap, ordenado por peso/relevância (1–10).
- Sistema de diálogos com NPCs via Árvore de Decisão com navegação por escolhas.

- Rota de investigação otimizada por TSP heurístico (vizinho mais próximo + 2-opt) e TSP exato (Held-Karp, $n \leq 15$) na Sprint 2.
- Tela de acusação final com exibição das 3 pistas mais relevantes (top3 do MaxHeap).
- HUD com barra de XP, level e inventário lateral.
- Backend REST com MongoDB Atlas, schemas Mongoose e seed completo.
- Testes unitários Jest com cobertura 100% nas estruturas de dados.

3.2 Fora do Escopo

- Sistema de autenticação de usuários.
- Multiplayer ou sessões simultâneas.
- Deploy em produção / hospedagem pública.
- Aplicativo desktop instalável (possível extensão futura via PWA ou Electron).
- Internacionalização (i18n).

4. Stack Tecnológica

Camada	Tecnologia	Justificativa
Frontend	React 18 + TypeScript + Vite	Componentização, tipagem estática, build rápido
Gerência de estado	Zustand	Store leve, sem boilerplate, acesso global
Roteamento	React Router DOM v6	Navegação declarativa entre telas
Backend	Node.js + Express	JavaScript full-stack, API REST simples e rápida
Banco de dados	MongoDB Atlas + Mongoose	Documento flexível para estado de partida; schemas tipados
Testes	Jest	Framework padrão para JavaScript, cobertura nativa
Controle de versão	Git + GitHub (Git Flow)	Branches por feature/issue, PRs para develop

5. Arquitetura do Sistema

O sistema segue uma arquitetura em três camadas, com separação estrita de responsabilidades:

5.1 Camada de Estruturas de Dados (Dev 1)

Módulos JavaScript puros, sem dependência de banco ou framework. São testados de forma completamente isolada via Jest. Cada estrutura implementa um algoritmo específico exigido pelas disciplinas.

Estrutura	Algoritmo	Complexidade	Papel no Jogo
MaxHeap.js	MaxHeap	insert $O(\log n)$ / top3 $O(\log n)$	Inventário de pistas ordenado por relevância
TSP.js	Vizinho Mais Próximo + 2-opt	$O(n^2)$ por iteração	Rota de investigação entre locais com pistas
ArvoreDecisao.js	Árvore de Decisão	get $O(1)$ via Map / escolher $O(1)$	Navegação nos diálogos dos NPCs
BFS.js	Busca em Largura	$O(V+E)$	Campo de visão do detetive no grid

5.2 Camada de Dados (Dev 2)

Schemas Mongoose para as coleções Partida, Pista e Suspeito. Seed reproduzível que popula o MongoDB Atlas com os dados do caso Blackwood: 10 pistas, 4 suspeitos e todos os nós de diálogo.

5.3 Camada de API (Dev 1)

Controllers e rotas Express que expõem 8 endpoints REST. Os controllers recebem requisições HTTP, delegam processamento às estruturas de dados e persistem/consultam o banco via Mongoose.

Endpoint	Método	Responsabilidade
/api/partida/:id/mover	POST	Move o detetive e retorna campo de visão (BFS)
/api/partida/:id/visao	GET	Retorna campo de visão atual
/api/partida/:id/inventario	GET	Retorna pistas ordenadas pelo MaxHeap
/api/partida/:id/coletar	POST	Insere pista no MaxHeap e persiste
/api/partida/:id/rota	GET	Calcula rota TSP com pistas pendentes
/api/partida/:id/acusar	POST	Executa top3 do heap e retorna veredicto
/api/partida/:id/interagir	POST	Inicia diálogo do NPC da célula

Endpoint	Método	Responsabilidade
/api/partida/:id/escolha	POST	Navega na Árvore de Decisão e acumula XP

5.4 Camada de Interface (Dev 3 + Dev 1)

Componentes React/TypeScript consumindo o gameStore (Zustand). O Dev 3 é responsável pelo mapa, coleta de pistas, diálogos e visualização TSP. O Dev 1 herdou o HUD, Inventário e tela de Acusação após saída do Dev 4 do projeto.

6. Requisitos Funcionais

6.1 Mecânica de Mapa e Movimento

- Grid 10×10 renderizado com CSS Grid, células de 48×48px.
- Movimento do detetive via WASD ou teclas de seta.
- Névoa de guerra: células além do raio de visão cobertas por overlay escuro.
- BFS calcula as células visíveis a cada movimento (raio padrão: 3 células).
- Animação de fade-in (300ms) ao revelar célula nova.
- Movimento bloqueado fora dos limites do grid e durante diálogos ativos.

6.2 Coleta de Pistas

- Ao entrar em célula com pista, coleta automática disparada.
- Pista inserida no MaxHeap por peso; inventário lateral atualizado em tempo real.
- Notificação deslizante exibida com nome e peso da pista coletada.
- Código de cores por relevância: peso ≥ 9 em vermelho, 7–8 em amarelo, < 7 em verde.

6.3 Sistema de Diálogos

- Ao entrar em célula com NPC, diálogo abre automaticamente.
- Texto exibido com efeito typewriter (30ms por caractere).
- Até 3 botões de escolha exibidos após conclusão do texto.
- Cada escolha pode desbloquear pistas e conceder XP ao jogador.
- Nó terminal exibe botão "Encerrar conversa".

6.4 Rota de Investigação (TSP)

- Botão "Sugerir Rota" chama GET /api/partida/:id/rota.
- Rota exibida como linha pontilhada conectando os locais pendentes na ordem retornada.
- Na Sprint 2: Held-Karp acionado automaticamente para $n \leq 15$.

6.5 Tela de Acusação

- Lista os suspeitos com nome, ícone e motivo resumido.
- Tela de confirmação antes de enviar a acusação.
- Backend retorna top3 do MaxHeap e monta argumento.
- Acerto: tela verde com confetes CSS. Erro: tela vermelha com animação shake.
- As 3 pistas usadas são exibidas com nome e peso.

7. Requisitos Não Funcionais

Requisito	Critério
Cobertura de testes	100% de Statements, Branches, Functions e Lines nas estruturas de dados
Tempo de resposta	Endpoints respondem em < 500ms em ambiente local
Compatibilidade	Funciona nos navegadores modernos (Chrome, Firefox, Edge)
Versionamento	Git Flow: branches por feature/issue, PRs para develop, merge na main ao encerrar sprint
Documentação	README técnico com instruções de instalação, arquitetura e análise de complexidade
Qualidade de código	Sem comentários desnecessários; inline notes apenas em lógica não óbvia

8. Divisão de Responsabilidades

Dev	Nome	Responsabilidades
Dev 1	Lucas da Silva Santos	Estruturas de dados (MaxHeap, TSP, ArvoreDecisao, BFS), testes Jest, controllers REST, rotas, server.js, HUD, Inventario, tela de Acusação (Issues #1, #2, #3, #7)
Dev 2	João David Salomão	Schemas Mongoose (Partida, Pista, Suspeito), seed MongoDB completo, README técnico, Held-Karp Sprint 2 (Issues #11, #9, #10)
Dev 3	Pollyana de Castro Rodrigues	Componentes de mapa, movimento, névoa de guerra, coleta de pistas, sistema de diálogos, visualização TSP no grid (Issues #4, #5, #6, #8)

9. Critérios de Conclusão

Sprint 1

- 24/24 testes Jest passando com cobertura 100% nas estruturas.
- Seed rodando sem erros e banco MongoDB populado com dados reais.
- Frontend jogável localmente: movimento, coleta, diálogos e acusação funcionando com dados mockados.
- Todas as branches mergeadas na develop sem conflitos.

Sprint 2

- Integração completa frontend ↔ backend ↔ banco de dados.
- Held-Karp implementado e integrado ao endpoint /rota.
- README técnico com justificativas algorítmicas e diagrama de arquitetura.
- Análise de complexidade documentada por algoritmo.
- Merge na main após aprovação do time.